



1

Chapitre 4: Javascript



Dorra DHAOU

Maître assistante en Informatique
Docteur et Ingénieur en Informatique

dorra.dhaou@fst.utm.tn / dorradhaou@gmail.com

AU. 2025 - 2026

1

Introduction

2

- Javascript est un langage de scripts, orienté objet et interprété.
- Il peut être intégré dans le code HTML des pages Web.
- Il permet d'améliorer leurs présentations et interactions avec les visiteurs.
- Le code javascript est interprété et exécuté côté client par le navigateur Web.
 - ▣ Le temps de réponse est assez réduit

2

Plan

3

- Exemple du javascript
- Intégration du javascript
- Entrées/sorties
- ...

3

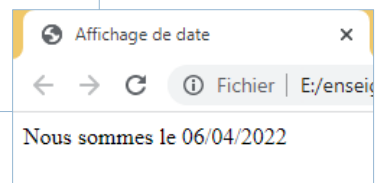
Exemple du javascript

4

Affichage de date

- Avec HTML, on ne peut créer que des pages Web statiques dont le contenu ne peut pas changer.

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    Nous sommes le 06/04/2022
  </body>
</html>
```



4

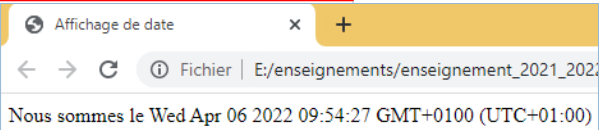
Exemple du javascript

5

Affichage de date

- Avec HTML et javascript, on peut créer des pages Web dynamiques dont le contenu peut changer.

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript">
      var date = new Date();
      document.write("Nous sommes le " + date);
    </script>
  </body>
</html>
```



5

6

Intégration du javascript

6

Intégration du javascript

7

- Le code javascript peut être intégré dans un document HTML avec quatre méthodes dans:
 - ▣ L'entête du document (balise **<head>**) en utilisant la balise **<script>**
 - ▣ Le corps du document (balise **<body>**) en utilisant la balise **<script>**
 - ▣ Un fichier externe « .js » en utilisant la balise **<script>** dans l'entête ou le corps du document
 - ▣ Un évènement d'un élément (clic, soumission, etc.)

7

Intégration du javascript

8

Méthode 1 - entête HTML

- La balise **<script>** permet de définir la fonction javascript dans l'entête du document HTML.
 - ▣ L'attribut **type** permet de définir le type du contenu (text/javascript par défaut)
- Exemple: affichage de date (définition)

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript">
      function afficherDate(){
        var date = new Date();
        return "Nous sommes le " + date;}
    </script>
  </head>

```

8

Intégration du javascript

9

Méthode 1 - entête HTML

- La balise `<script>` permet d'appeler la fonction javascript déclarée dans l'entête (balise `<head>`) dans le corps du document HTML (balise `<body>`).
- Exemple: affichage de date (appel)

```
<body>
  <script>
    document.write(afficherDate());
  </script>
</body>
</html>
```

9

Intégration du javascript

10

Méthode 1 - entête HTML

- Exemple: affichage de date (définition + appel)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript">
      function afficherDate(){
        var date = new Date();
        return "Nous sommes le " + date;
      }
    </script>
  </head>
  <body>
    <script>
      document.write(afficherDate());
    </script>
  </body>
</html>
```

Affichage de date x +

← → ↻ ⓘ Fichier | E:/enseignements/enseignement_2021_2022

Nous sommes le Wed Apr 06 2022 10:26:25 GMT+0100 (UTC+01:00)

10

Intégration du javascript

11

Méthode 2 - corps HTML

- La balise **<script>** permet d'appeler une fonction ou exécuter une commande javascript dans le corps du document HTML.
- Exemple: affichage de date

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript">
      var date = new Date();
      document.write("Nous sommes le " + date);
    </script>
  </body>
</html>

```

11

Intégration du javascript

12

Méthode 3 – fichier externe

- La balise **<script>** permet d'intégrer du code javascript écrit dans un fichier externe.
- Elle peut être insérée dans:
 - ▣ L'entête du document HTML pour une exécution **différée**
 - ▣ Le corps du document HTML pour une exécution **directe**
- L'attribut **src** permet de spécifier le fichier externe à intégrer.

12

Intégration du javascript

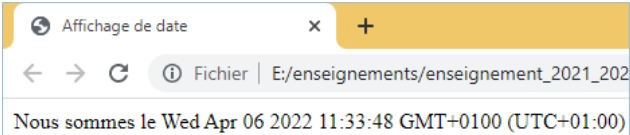
13

Méthode 3 – fichier externe

- Exemple: affichage de date (exécution différée)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript" src = "script.js"> </script>
  </head>
  <body>
    <script>
      document.write(afficherDate());
    </script>
  </body>
</html>
```

```
function afficherDate() {
  var date = new Date();
  return "Nous sommes le " + date;
}
```



13

Intégration du javascript

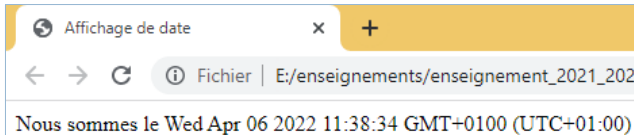
14

Méthode 3 – fichier externe

- Exemple: affichage de date (exécution directe)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script2.js"> </script>
  </body>
</html>
```

```
var date = new Date();
document.write("Nous sommes le " + date);
```



14

Intégration du javascript

15

Méthode 4 – évènement d'un élément HTML

- Il existe plusieurs évènements relatifs aux éléments HTML (clic, soumission, réinitialisation, etc.).
- Dans une balise HTML, un évènement est identifié par un attribut (**onclick**, **onsubmit**, **onreset**, etc.).
- La valeur de chaque attribut doit comprendre le code javascript à exécuter une fois l'évènement réalisé.

15

Intégration du javascript

16

Méthode 4 – évènement d'un élément HTML

- Exemple: affichage de date

The screenshot shows an HTML editor on the left and a web browser on the right. The editor contains the following code:

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript" src = "script3.js">
    </script>
  </head>
  <body>
    <input type = "button" value = "Afficher la date" onClick = "afficherDate()"/>
  </body>
</html>

```

The browser window shows the title "Affichage de date" and a button labeled "Afficher la date". The console output shows the date: "Nous sommes le Wed Apr 06 2022 11:59:32 GMT+0100 (UTC+01:00)".

16

17

Entrées/sorties

17

Entrées/sorties

18

- En javascript, trois fonctions peuvent être utilisées pour afficher une boîte de dialogue aux visiteurs:
 - ▣ **alert()**
 - Afficher un message aux visiteurs avec un bouton « **OK** »
 - ▣ **confirm()**
 - Fournir un choix aux visiteurs avec deux boutons « **OK** » et « **Annuler** »
 - ▣ **prompt()**
 - Permettre aux visiteurs de saisir un message, une valeur, etc. avec deux boutons « **OK** » et « **Annuler** »

18

Entrées/sorties

19

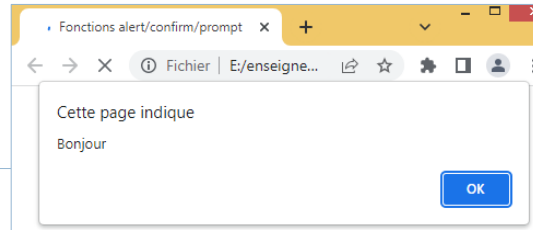
Fonction alert()

- Le message à afficher aux visiteurs doit être passé comme paramètre de la fonction **alert()**.

- Exemple:

```
alert("Bonjour");
```

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm/prompt </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
  </body>
</html>
```



19

Entrées/sorties

20

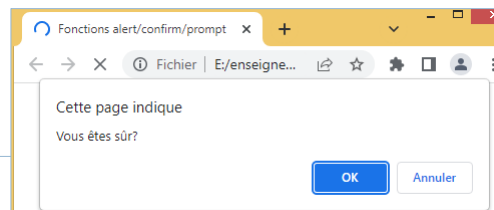
Fonction confirm()

- Le message à afficher aux visiteurs doit être passé comme paramètre de la fonction **confirm()**.

- Exemple:

```
confirm("Vous êtes sûr?");
```

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm/prompt </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
  </body>
</html>
```



20

Entrées/sorties

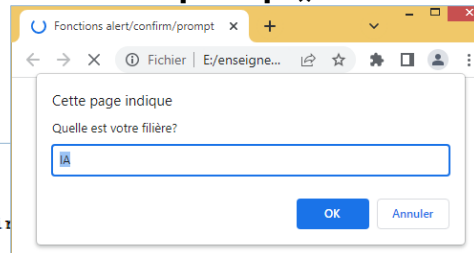
21

Fonction prompt()

- Le message à afficher aux visiteurs et le texte par défaut du champ de saisie doivent être passés comme paramètres de la fonction **prompt()**.

- Exemple:

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
    <script>
      prompt("Quelle est votre filière?", "IA");
    </script>
  </body>
</html>
```



21

Entrées/sorties

22

Exercice 1

- Écrire un script javascript permettant de demander le nom et l'âge du visiteur et les afficher dans une boîte de dialogue séparés par une virgule.

```
var nom = prompt("Donner votre nom:");
var age = prompt("Donner votre âge:");
var msg = nom + ',' + age;
alert(msg);

<!doctype html>
<html>
  <head>
    <title> Exercice 1 </title>
    <meta charset = "UTF-8"/>
  </head>
  <body>
    <script type = "text/javascript" src = "script_exercice1.js"> </script>
  </body>
</html>
```

22

Entrées/sorties

23

Exercice 2

- Écrire un script javascript permettant de demander la saisie de deux entiers et d'afficher la somme.

```

<!doctype html>
<html>
  <head>
    <title> Exercice 2 </title>
    <meta charset = "UTF-8"/>
  </head>
  <body>
    <script type = "text/javascript" src = "script_exercice2.js"> </script>
  </body>
</html>

```

```

var a = prompt("Donner un entier:");
var b = prompt("Donner un entier:");
var res = parseInt(a)+parseInt(b);
alert("La somme = "+res);

```

23

La Console JavaScript

24

- La console est un outil intégré au navigateur (accessible via la touche **F12** ou **Clic droit > Inspecter**).
- Elle est indispensable pour le développeur car elle permet de :

A. Déboguer (Contrôler le contenu)

- Elle permet d'afficher la valeur des variables à un instant précis pour vérifier si le programme récupère les bonnes données.
- On utilise la commande : `console.log(maVariable);`

B. Tracer (Suivre l'exécution)

- Elle permet de vérifier quels blocs de code (conditions, boucles, fonctions) sont réellement visités par le programme.
- Cela permet de comprendre le cheminement logique du script et de repérer où l'exécution s'arrête.

24

La Console JavaScript

25

Pourquoi utiliser la console ?

- **Gain de temps** : On évite d'utiliser trop d'`alert()` qui bloquent l'interaction utilisateur.
- **Précision** : Elle affiche les erreurs de syntaxe précises et la ligne exacte où le code a échoué.
- **Discrétion** : Les messages envoyés à la console ne sont pas visibles par le visiteur du site, seulement par le développeur.

25

La Console JavaScript

26

Exécution hors navigateur (NodeJS)

- Il est possible d'exécuter du code JavaScript sans navigateur en utilisant **NodeJS**.
- **NodeJS** est un environnement d'exécution qui permet d'utiliser JavaScript côté serveur ou sur votre propre machine.

Comment tester son programme ?

- Ouvrir une console (Invite de commandes, PowerShell ou Cmdr).
- Se placer dans le dossier contenant votre fichier.
- Lancer la commande : `node nomFichier.js`

26

La Console JavaScript

27

□ **Exemple:** fichier file.js

```
function maFonction() {  
  console.log("Hello World !");  
}  
maFonction();
```

→ Lancer la commande `node file.js`

Résultat affiché dans la console : Hello World !

□ **Il est aussi possible de définir un raccourci de** `console.log`

```
var c1 = console.log;  
c1("Hello World !");
```

27

28

Concepts de base

28

Variables

29

- En JavaScript, on peut déclarer des variables avec trois mots-clés :
 - ▣ **var** : Ancienne syntaxe pour déclarer une variable (portée de fonction).
 - ▣ **let** : Syntaxe moderne pour déclarer une variable dont la valeur peut changer (portée de bloc).
 - ▣ **const** : Utilisé pour déclarer une constante dont la valeur ne peut pas être modifiée après affectation.

Exemples :

```
var nom = "Ali";
let age = 25;
const pays = "Tunisie";
```

29

Portée des variables déclarées

30

- Une variable déclarée avec le mot-clé **var** peut avoir une **portée globale**.

```
{
  var x = 1;
}
console.log(x); // affiche : 1
```

- Même si la variable **X** est déclarée à l'intérieur d'un bloc **{ }**, elle reste **accessible à l'extérieur**. (**X** reste accessible **en dehors du bloc { }**)

Remarque

- En JavaScript, une variable déclarée dans un bloc **{ ... }** (comme dans **if**, **for**, etc.) avec **var** a une **portée globale ou de fonction**, et non une portée limitée au bloc.

30

Portée des variables déclarées

31

- Une variable déclarée avec le mot-clé **let** possède une **portée limitée au bloc { ... }** dans lequel elle est déclarée.

```
{
  let x = 1;
}
console.log(x); // affiche : erreur : x is not defined
```

- Dans cet exemple, la variable **x** n'est accessible qu'à l'intérieur du bloc.

Remarque

- En JavaScript, Une variable déclarée avec **let** dans un bloc (if, for, {...}) n'est pas accessible en dehors de ce bloc.

31

Portée des variables déclarées

32

- Une variable déclarée avec **const** possède également une **portée limitée au bloc { ... }**.

```
{
  const y = 5;
}
console.log(y); // affiche : erreur : y is not defined
```

- Dans cet exemple, la variable **y** n'est accessible qu'à l'intérieur du bloc.

Remarque

- En JavaScript, **const** empêche la modification de la valeur après son initialisation/affectation.
- Comme **let**, sa portée est limitée au bloc.

32

Permuter le contenu de deux variables

33

- En JavaScript, on peut **échanger les valeurs de deux variables** grâce à la **déstructuration** :

```
let a = 2;
let b = 0;
[a, b] = [b, a];
console.log(a); // 0
console.log(b); // 2
```

- La syntaxe `[a, b] = [b, a]` permet de **permuter les valeurs sans utiliser de variable temporaire**.

33

Types de données

34

- JavaScript utilise plusieurs types de données fondamentaux :
 - ▣ **Number** : Pour les nombres (entiers ou décimaux).
 - ▣ **String** : Pour les chaînes de caractères (entre " " ou ' ').
 - ▣ **Boolean** : Pour les valeurs logiques (true ou false).
 - ▣ **Object** : Pour stocker des collections de données.
 - ▣ **Array** : Pour stocker des listes d'éléments.

Exemples :

```
let score = 95;           // Number
let message = "Bravo";    // String
let estValide = true;     // Boolean
```

34

Opérateur typeof

35

- **Récupérer le type de la valeur stockée dans une variable**
- En JavaScript, l'opérateur **typeof** permet de connaître le **type d'une valeur ou d'une variable**.

```
console.log(typeof 5);           // number
console.log(typeof true);        // boolean
console.log(typeof 5.2);         // number
console.log(typeof "bonjour");   // string
console.log(typeof 'c');         // string
var x;
console.log(typeof x);           // undefined
console.log(typeof {nom:"Ben foulén", prenom:"Foulén"}); // object
console.log(typeof new Date()); // object
console.log(typeof Date);        // function
console.log(typeof [1,2,3,4]);   // object
console.log(typeof null);        // object
```

- ➔ Pour les **objets prédéfinis en JavaScript**, **typeof** peut retourner deux types principaux : **object** (si c'est une **instance d'objet**) ou **function** (**constructeur ou fonction normale**)

35

Opérateurs arithmétiques

36

- L'opérateur **+** possède deux fonctions en JavaScript :
- **L'addition numérique** : si les deux opérandes sont des nombres.
- **La concaténation** : si l'un des opérandes est une **chaîne de caractères** (string), JavaScript transforme l'autre en chaîne et les "colle" ensemble.

Remarques :

- **La priorité de gauche à droite.**
- **Le type "String" gagne toujours** : Dès qu'une chaîne apparaît dans une suite d'additions, le reste du résultat devient souvent une chaîne.
- **NaN (Not a Number)** : Apparaît lorsqu'on essaie de faire une opération mathématique avec une variable déclarée mais non initialisée.

36

Opérateurs arithmétiques

37

```

var x = 1;
y = 3;
z = '8';
t = "2";

u = "bonjour";
v = "3bonjour";
var m;
n = 2.5;

console.log(x + y); // 1 + 3 = 4
console.log(x + z); // 1 + '8' → '18' (concaténation car '8' est une chaîne)
console.log(x + t); // 1 + "2" → "12"
console.log(x + y + z); // (1 + 3) + '8' → 4 + '8' → "48"
console.log(x + u); // 1 + "bonjour" → "1 bonjour"
console.log(x + v); // 1 + "3bonjour" → "13bonjour"
console.log(u + v); // "bonjour" + "3bonjour" → "bonjour3bonjour"
console.log(x + m); // 1 + undefined → NaN (Not a Number)
console.log(x + n); // 1 + 2.5 = 3.5

```

37

Opérateurs arithmétiques

38

□ En plus de l'addition (+), JavaScript utilise les opérateurs suivants :

- * : Multiplication
- - : Soustraction
- / : Division
- % : Modulo (reste de la division entière)
- ** : Exposant (ex: 2**3 donne 8)

Quelques raccourcis d'écriture (Incrémentations) :

- i++; équivaut à i = i + 1;
- i--; équivaut à i = i - 1;
- i += 2; équivaut à i = i + 2;
- i *= 3; équivaut à i = i * 3;

38

L'opérateur unaire + et -

39

- L'opérateur **unaire +** permet de **convertir une valeur en nombre**.

```
console.log(typeof (+ "3")); // affiche : number
console.log(+true);         // affiche : 1
```

- "3" (chaîne de caractères) devient le nombre 3.
- true devient 1, et false deviendrait 0.

- L'opérateur **unaire -** sert à **négaliver une valeur** ou à la **convertir en nombre négatif**.

```
console.log(typeof (- "3")); // affiche : number
console.log(- "3");         // affiche : 3
console.log(-true);         // affiche : -1
```

- "-3" → -3, et ensuite -(-3) → 3.
- true devient 1, donc -true → -1.

39

La fonction eval()

40

- La fonction **eval()** permet d'évaluer une **expression arithmétique écrite sous forme de chaîne de caractères** :

```
var str = "2 + 5 * 3";
console.log(eval(str)); // affiche : 17
```

Attention :

- si l'expression est mal formulée, **eval()** génère une **EvalError**.
- **eval()** exécute tout code JavaScript contenu dans la chaîne, donc son utilisation doit être prudente.

40

Déclaration des chaînes de caractères

41

- En JavaScript, une chaîne de caractères peut être déclarée de **deux façons différentes** :

```
var str1 = new String("bonjour");
console.log(typeof(str1)); // affiche : object

var str2 = "bonjour";
console.log(typeof(str2)); // affiche : string
```

- Avec **new String()** : on crée un objet String, donc typeof retourne object.
- Avec **"..."** : on crée une chaîne de caractères simple, donc typeof retourne string.

41

Méthodes utiles pour les chaînes de caractères

42

- En JavaScript, les chaînes de caractères disposent de plusieurs **méthodes pratiques** :

Méthode	Description
length	Retourne la longueur de la chaîne
toUpperCase()	Convertit la chaîne en majuscules
toLowerCase()	Convertit la chaîne en minuscules
trim()	Supprime les espaces au début et à la fin
substr(i, j)	Extrait une sous-chaîne à partir de l'indice i sur j caractères
indexOf(sub)	Retourne la position de la sous-chaîne sub dans la chaîne, ou -1 si elle n'existe pas
...	D'autres méthodes existent (ex : slice(), replace(), includes(), ...)

42

Accéder à un caractère dans une chaîne

43

□ Il existe **plusieurs façons** d'accéder à un caractère à l'indice *i* :

1. **Via l'indice directement** : `console.log(str[i]);`
2. **En extrayant une sous-chaîne** :
`console.log(str.substr(i, 1));` // extrait 1 caractère à partir de l'indice *i*
3. **Avec la méthode `charAt()`** : `console.log(str.charAt(i));`

Remarque

- Si vous appelez **une méthode sur une variable undefined**, JavaScript génère une **erreur `TypeError`**.

```
var str;
console.log(str.toUpperCase()); // TypeError : Cannot read property
                                'toUpperCase' of undefined
```

43

Structures de contrôle conditionnelles

44

Les conditions permettent de modifier le comportement du script selon des critères logiques.

- **La structure `if / else`** :
 - **`if`** : exécute un bloc si la condition est vraie.
 - **`else if`** : teste une nouvelle condition si la précédente est fausse.
 - **`else`** : s'exécute si aucune condition n'est remplie.

Exemple :

```
let score = 75;

if (score > 50) {
  alert("Résultat : Réussi");
} else {
  alert("Résultat : Échec");
}
```

44

Structures de contrôle conditionnelles

Opérateurs de comparaison

45

- Pour formuler les conditions, on utilise des opérateurs de comparaison et des opérateurs logiques :

- **Opérateurs logiques** : && : et

|| : ou

! : non

- **Opérateurs de comparaison** :

Opérateur	Description	Exemple
==	Égal à (valeur)	5 == "5" (Vrai)
===	Égal à (valeur ET type)	5 === "5" (Faux)
!= / !==	Différent de	a != b
>, <	Supérieur / Inférieur	score > 10
>=, <=	Supérieur ou égal...	age >= 18

45

Structures de contrôle conditionnelles

46

Les conditions permettent de modifier le comportement du script selon des critères logiques.

- **La structure switch** : idéale pour tester plusieurs valeurs d'une même variable.

C'est une alternative plus lisible au if/else if lorsqu'on doit comparer une même variable à plusieurs valeurs possibles.

- Mots-clés :

- **case** : définit une valeur à tester.
- **break** : arrête l'exécution et sort du bloc (très important).
- **default** : s'exécute si aucun cas ne correspond (optionnel).

46

Structures de contrôle conditionnelles

47

□ La structure switch :

Exemple :

La variable dans **switch** peut être :

- Un nombre
- Un caractère
- Une chaîne de caractères (contrairement aux java, C, C++ ...)

```
let jour = 2;
let nomJour;

switch (jour) {
  case 1:
    nomJour = "Lundi";
    break;
  case 2:
    nomJour = "Mardi";
    break;
  case 3:
    nomJour = "Mercredi";
    break;
  default:
    nomJour = "Jour inconnu";
}

alert("Aujourd'hui nous sommes : " + nomJour);
// Affiche "Mardi"
```

47

Structures de contrôle conditionnelles

48

□ Considérons l'exemple suivant :

```
var message = prompt('Saisir un message');
switch (message) {
  case "bonjour":
    alert('salut !');
    break;
  default:
    alert('quoi ?');
}
```

□ Version simplifiée avec l'opérateur ternaire

```
var message = prompt('Saisir un message');
message == "bonjour" ? alert("salut !") : alert('quoi ?');
```

➔ Le **ternaire** permet d'écrire un **if/else** en une seule ligne :

condition ? action_si_vrai : action_si_faux

- **switch** est pratique pour plusieurs cas possibles.
- Le **ternaire** est plus concis pour des tests simples à deux options.

48

Structures de contrôle itératives : Les boucles (loops)

49

- Les boucles servent à répéter un bloc d'instructions tant qu'une condition est vérifiée.

- **Types de boucles :**

- **for** : utilisée quand on connaît à l'avance le nombre de répétitions.
 - **while** : Répéter tant qu'une condition est vraie (test au début).
 - **do...while** : Exécuter au moins une fois, puis répéter tant que la condition est vraie. (*répéter... jusqu'à* en algorithmique)

- **Exemples :**

```
// Boucle FOR : affiche les nombres de 0 à 4
for (let i = 0; i < 5; i++) {
  console.log("Compteur : " + i);
}
```

```
// Boucle WHILE
let j = 0;
while (j < 3) {
  console.log("Passage n°" + j);
  j++;
}
```

49

Programmation modulaire Fonctions

50

- Une fonction est un bloc de code réutilisable :

Déclaration : **function** nomFonction(params) {

Les instructions de la fonction

..... }

Appel : nomFonction(valeur)

Exemples :

```
function saluer(nom) {
  alert("Bonjour " + nom);
}

saluer("Ali"); // Appel de la fonction
```

```
function somme(a, b) {
  return a + b;
}

var resultat = somme(1, 3);
console.log(resultat);
```

50

Programmation modulaire

Fonctions

51

- **Une fonction anonyme** est une fonction qui n'a pas de nom propre. On l'affecte généralement à une **variable** pour pouvoir l'utiliser plus tard.
- **Déclaration:** `Var nomVariable = function (params) {`
Les instructions de la fonction
`..... }`

Exemple :

```
var somme2 = function(a, b) {
  return a + b;
};
```

```
var resultat2 = somme2(1, 3); // resultat2 vaudra 4
```

51

Programmation modulaire

Fonctions

52

- En JavaScript moderne, on utilise de moins en moins `var`. Il est préférable d'utiliser **const** pour les fonctions, car leur définition ne change généralement pas.
- De plus, on utilise souvent les **fonctions fléchées (Arrow Functions)** pour plus de concision :

```
const somme2 = (a, b) => a + b;
```

- On retire le mot-clé `function`, les accolades `{ }` et `return` et on ajoute une **flèche**.

52

Programmation modulaire

Fonctions

53

- Différentes façons d'écrire la fonction

Style	Syntaxe	Usage recommandé
Classique	function somme(a, b) { ... }	Pour des fonctions complexes ou nommées.
Fléchée	const somme = (a, b) => a + b;	Pour la concision et les fonctions anonymes.
Anonyme	const somme = function(a, b) { ... };	Moins fréquent aujourd'hui.

53

Programmation modulaire

Fonctions

54

- Une autre méthode très particulière existe pour créer une fonction, car elle utilise le constructeur **new Function()**.
- Contrairement aux autres méthodes, ici tout est écrit sous forme de **chaînes de caractères** (entre guillemets).

- **Syntaxe** :

```
var nomFonction = new Function("arg1", "arg2", "corps de la fonction");
```

- **Exemple** :

```
var somme4 = new Function ("a", "b", "return a + b");
```

```
//Appel
```

```
var res = somme4 (1,3)
```

54

Programmation modulaire

Fonctions

55

➤ Une différence technique importante : la portée

Contrairement aux fonctions classiques, une fonction créée avec **new Function** ne peut pas accéder aux variables locales de l'endroit où elle est créée. Elle s'exécute toujours dans le **contexte global**.

❑ Exemple :

```
let x = 10;
function tester() {
  let x = 20;
  // Cette fonction cherchera x dans le global, pas dans tester()
  let f = new Function("return x");
  return f();
}
console.log(tester()); // Affichera 10 (global), pas 20 !
```

55

Les tableaux

56

❑ On peut créer un tableau en **deux façons** :

1. Avec le constructeur Array

```
var tab = new Array(value1, value2, ..., valueN);
```

2. Avec la syntaxe littérale (raccourci)

```
var tab = [value1, value2, ..., valueN];
```

❑ Accès à un élément

Pour accéder à un élément du tableau à l'indice i :

```
tab[i];
```

56

Les tableaux

57

□ Opérations de base sur les tableaux

Méthodes	Description
push(elem)	Ajoute un élément à la fin du tableau
pop()	Supprime le dernier élément
shift()	Supprime le premier élément
indexOf(elem)	Retourne la position d'un élément, ou -1 si absent
reverse()	Inverse l'ordre des éléments
sort()	Trie le tableau
splice(start, deleteCount, ...)	Extrait, ajoute, remplace ou supprime un ou plusieurs éléments tout en modifiant le tableau d'origine.
slice(start, end)	Extrait un sous-tableau sans modifier l'original
...	Autres méthodes disponibles

57

Les tableaux

58

- **tableau.splice** (*indexDepart*, *nbASupprimer*, *elem1*, *elem2*, ...);
 - **indexDepart** : La position où l'on commence (0 étant le premier élément).
 - **nbASupprimer** : Le nombre d'éléments que l'on veut retirer.
 - **elem1, elem2...** (optionnel) : Les nouveaux éléments que l'on veut insérer à partir de l'index de départ.
- Pour **ajouter sans rien supprimer**, il faut mettre le deuxième argument (*nbASupprimer*) à 0.
- Pour **supprimer** (sans ajouter), il suffit de préciser l'index de départ et le nombre d'éléments à retirer, sans ajouter d'autres arguments à la fin:

tableau.**splice**(*indexDepart*, *nbASupprimer*)
- Pour **remplacer**, Il faut combiner les deux : il suffit d'indiquer combien d'éléments à enlever, et lister les nouveaux éléments qui prendront leur place :

tableau.**splice**(*indexDepart*, *nbASupprimer*, *nouvelElement*)

58

Les tableaux

Exemples avec splice ():

59

```
const couleurs = ["Bleu", "Rouge"];
// On commence à l'index 1, on ne supprime rien, et on ajoute Vert et Jaune
couleurs.splice(1, 0, "Vert", "Jaune");
console.log(couleurs); // Résultat : ["Bleu", "Vert", "Jaune", "Rouge"]

-----

const Tab = ["A", "B", "C", "D"];
// Supprimer 2 éléments à partir de l'index 1 (B et C)
Tab.splice(1, 2);
console.log(Tab); // Résultat : ["A", "D"]

-----

const voitures = ["Tesla", "Renault", "Peugeot"];
// Remplacer "Renault" (index 1) par "BMW"
// On supprime 1 élément à l'index 1, et on insère "BMW"
voitures.splice(1, 1, "BMW");
console.log(voitures); // Résultat : ["Tesla", "BMW", "Peugeot"]
```

59

Les tableaux

60

slice(start, end) : Extrait un sous-tableau sans modifier l'original

- start est **inclus** → l'élément à l'indice start **fait partie** du sous-tableau.
- end est **exclus** → l'élément à l'indice end **ne fait pas partie** du sous-tableau.

□ Exemples avec slice

```
var sports = ["foot", "tennis", "basket", "volley"];
tab = sports.slice(1, 3);
console.log(sports); // affiche : foot, tennis, basket, volley
console.log(tab); // affiche : tennis, basket

-----

var tab = ["a", "b", "c", "d", "e"];
var sousTab = tab.slice(1, 4);
console.log(sousTab); // affiche : ["b", "c", "d"]
```

60

Les tableaux

61

□ Autres opérations avancées (ES5+):

Méthodes	Description
forEach()	Parcourt un tableau et exécute une fonction pour chaque élément
map()	Applique une fonction à chaque élément et retourne un nouveau tableau
filter()	Retourne un tableau avec les éléments qui satisfont une condition
reduce()	Réduit le tableau à une seule valeur selon une fonction de cumul
...	D'autres méthodes disponibles

- Ces méthodes permettent de **manipuler facilement les tableaux** sans utiliser des boucles classiques (for).
- On travaille avec des fonctions au lieu d'écrire beaucoup de code répétitif.

61

Les tableaux

62

□ **forEach()** → parcourir un tableau

- Sert uniquement à **parcourir et afficher / traiter** les éléments
- **Ne retourne rien**

```
var tab = [2, 3, 5];
tab.forEach(function(x) {
  console.log(x); // affiche 2, 3, 5
});
```

□ **map()** → transformer un tableau

- Applique une fonction à chaque élément
- Retourne un **nouveau tableau**

```
var tab = [2, 3, 5];
var res = tab.map(function(x) {
  return x + 3;
});
console.log(res); // affiche [5, 6, 8]
```

62

Les tableaux

63

□ **filter()** → filtrer les éléments

- Garde seulement les éléments qui respectent une condition.
- Retourne **un nouveau tableau**

```
var tab = [2, 3, 5];
var res = tab.filter(function(x) {
    return x > 3;
});
console.log(res); // affiche [5]
```

□ **reduce()** → réduire à une seule valeur

- Combine tous les éléments pour produire une seule valeur.
- Utilisé pour somme, moyenne, produit...
- Retourne une **seule valeur**.

```
var tab = [2, 3, 5];
var somme = tab.reduce(function(sum, x) {
    return sum + x;
}, 0);
console.log(somme); // affiche 10
```

63

Les tableaux

64

□ **Exemple combiné : map, filter, forEach**

```
var tab = [2, 3, 5];
tab.map(x => x + 3)    // ajoute 3 à chaque élément
  .filter(x => x > 5)  // garde les éléments supérieurs à 5
  .forEach(function(a) {
    console.log(a - 3); // affiche 3 et 5
  });
```

□ **Fonction fléchée vs fonction classique**

// fléchée	// équivalent classique
x => x + 3	function(x) {
	return x + 3;
	}

64

Les tableaux

65

□ Exemple avec reduce

```
var tab = [2, 3, 5];
var somme = tab.map(x => x + 3)      // [5,6,8]
                .filter(x => x > 5)  // [6,8]
                .reduce(function(sum, elem){
                    return sum + elem;
                });

console.log(somme); // affiche : 14
```

65

Les tableaux

66

□ Parcours simplifié d'un tableau avec **for ... of**

```
var tab = [2, 3, 5];
for (var elt of tab) {
    console.log(elt);
}

// affiche : 2, 3, 5
```

□ Version avec for classique

```
for (var i = 0; i < tab.length; i++) {
    console.log(tab[i]);
}
```

66

Les objets en JavaScript

67

□ Problème :

Comment regrouper plusieurs informations dans une seule variable ?

Nom	Ben Foulen
Prénom	Foulen
Âge	20

✗ Mauvaise solution : tableaux car ils stockent des valeurs sans nom

✓ Bonne solution : **objets** car ils stockent des données avec des clés

67

Les objets en JavaScript

Qu'est-ce qu'un **objet** ?

68

□ Un objet est une structure de données qui regroupe des informations liées. Il se compose de paires (**clé : valeur**)

- **clé** → nom de l'attribut
- **valeur** → donnée

□ Un objet peut contenir :

- des **attributs** : (exple : nom, prenom, age)
- des **méthodes** : (exple: presenter())

□ Exemple:

```
const Etudiant = {
  nom: "Ben foulen",
  prenom: "foulen",
  age: 21,
  presenter: function() {
    return "Je m'appelle " + this.prenom;
  }
};
```

68

Les objets en JavaScript

69

□ Accès aux données d'un objet :

Il existe deux notations pour lire ou modifier une donnée :

- **Notation par point (.)** : La plus utilisée et lisible.
 - ▣ `console.log(Etudiant.nom); // Ben foulén`
- **Notation par crochets ([])** : Indispensable si le nom de la propriété est dynamique (stocké dans une variable).
 - ▣ `console.log(Etudiant["age"]); // 21`

□ Appel de méthode : `console.log(Etudiant.presenter());`

□ Parcourir un objet :

▪ Boucle sur un objet :

```
for (let key in Etudiant) {
  console.log(key + " : " + Etudiant[key]);
}
```

69

Les objets en JavaScript

70

□ Manipulation Dynamique : JavaScript est flexible, on peut modifier un objet après sa création.

- **Ajouter** : `Etudiant.note = 15;`

```
Etudiant.saluer = function() {
  console.log("Bonjour " + this.prenom);
};
```
- **Supprimer** : `delete Etudiant.age;`
- **Vérifier l'existence** : `"nom" in Etudiant; //(renvoie true)`

70

Les objets en JavaScript

71

Application :

- Ajouter une propriété moyenne, puis modifier la méthode présenter pour afficher :

"Je m'appelle foulén, j'ai 21 ans et ma moyenne est 15."

```
Etudiant.moyenne = 15;
```

```
presenter: function() {
    return "Je m'appelle " + this.nom +
        ", j'ai " + this.age +
        " ans et ma moyenne est " + this.moyenne;
}
```

71

Les objets en JavaScript

72

- **Le constructeur :** Utilise une fonction pour créer plusieurs objets identiques.

```
function Etudiant(nom, prenom, age) {
    this.nom = nom;
    this.prenom = prenom;
    this.age = age;
}

var e1 = new Etudiant("Youssef", "Ali", 20);
var e2 = new Etudiant("Mahjoub", "Sara", 18);
```

72

Les objets en JavaScript

73

- Une fois le **constructeur déclaré**, on ne peut pas ajouter un **attribut partagé** comme ceci :

```
Etudiant.ville = "Tunis"; // ✗ ne fonctionne pas comme attendu
```

- Cette écriture ajoute ville au constructeur lui-même mais **PAS aux objets créés**

```
var e1 = new Etudiant("Youssef", "Ali", 20);
console.log(e1.ville); // undefined ✗
```

- ➔ La solution est d'utiliser le **prototype** : un espace **partagé** par tous les objets créés avec un constructeur.

```
Etudiant.prototype.ville = "Tunis";
console.log(e1.ville); // Tunis ✓
console.log(e2.ville); // Tunis ✓
```

73

Les objets en JavaScript

74

- Pareil pour les méthodes / Getters et Setters :

```
Etudiant.prototype.saluer = function() {
    return "Bonjour " + this.prenom;
};

Etudiant.prototype.getAge = function() {
    return this.age;
};

Etudiant.prototype.setAge = function(age) {
    if (age > 0) {
        this.age = age;
    }
};

console.log(e1.getAge()); // 20
e1.setAge(35);
console.log(e1.getAge()); // 35
```

74

Objets et méthodes prédéfinies

75

□ Objets prédéfinis :

- `Math` : contenant de méthodes permettant de réaliser des opérations mathématiques sur les nombres.
- `Date` : permet de manipuler des dates
- `String` : permet de manipuler des chaînes de caractères (texte).
- `Array` : utilisé pour stocker des listes de données ordonnées.
- `JSON` : indispensable pour le transfert de données sur le web. Il est utilisé pour échanger des données avec un serveur ou une base de données.
- ...

75

Objets et méthodes prédéfinies

76

□ Exemple de méthodes de l'objet `Math`

```
Math.round(2.1); // retourne 2
Math.round(2.9); // retourne 3
Math.pow(2, 3); // retourne 8
Math.sqrt(4); // retourne 2
Math.abs(-2); // retourne 2
Math.min(0, 1, 4, 2, -4, -5); // retourne -5
Math.max(0, 1, 4, 2, -4, -5); // retourne 4
Math.random(); // retourne un nombre aléatoire entre [0,1[
Math.floor(Math.random() * 10); // retourne un entier
aléatoire arrondi à l'entier inférieur et qui est compris
entre 0 et 9
```

76

Objets et méthodes prédéfinies

77

□ L'objet Date

▪ Créer et afficher un objet date :

```
var date = new Date ();
console.log(date); // affiche la date complète du jour
```

▪ Exemples de méthodes pour l'objet Date :

```
getFullYear() : retourne l'année
getMonth() : retourne le mois [0-11]
getDate() : retourne le jour [1-31]
getHours() : retourne l'heure [0-23]
getMinutes() : retourne les minutes [0-59]
getSeconds() : retourne les secondes [0-59]
getMilliseconds() : retourne les millisecondes [0-999]
getTime() : retourne le nombre de millisecondes depuis le 1
             Janvier 1970 (Avec ES5, on peut utiliser Date.now())
```

77

78

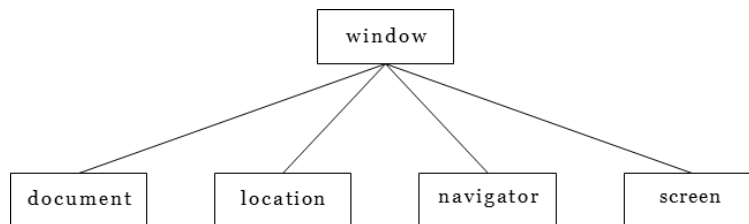
JavaScript et le DOM (Document Object Model)

78

Introduction

79

- Le langage JavaScript ne sert pas qu'à faire des calculs ; il agit comme un pont pour obtenir des informations sur l'environnement de l'utilisateur via des **objets prédéfinis** :
 - **Le Navigateur** : Connaître le nom, la version ou la langue (Objet navigator).
 - **Le Visiteur** : Analyser ses préférences ou son état de connexion.
 - **L'Écran** : Adapter l'affichage selon la taille ou l'orientation (Objet screen).
 - **La Page (Document)** : Manipuler le contenu HTML et la structure (Objet document).
 - **L'URL** : Connaître l'adresse actuelle ou rediriger l'utilisateur (Objet location).

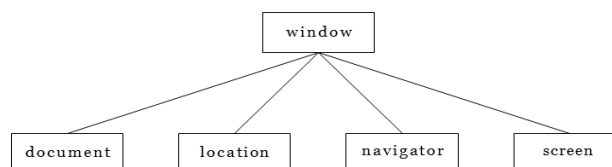


79

Introduction

80

- L'objet global **window** est la racine de tout en JavaScript côté client. Il contient :
 - quatre autres objets importants : **document**, **location**, **navigator**, **screen**
 - des méthodes de dialogue basiques comme `alert()`, `confirm()`, `prompt()`
 - les objets comme `String`, `Date`, `Math`...
 - toute variable, objet... définis par le développeur
 - ...



80

Introduction

81

□ Les sous-Objets clés :

- **document** : Représente la page HTML (DOM). Contient le `contentType` (ex: `text/html`) et toutes les balises.
- **location** : Gère l'adresse URL. Exemples : `hostname` (`localhost`), `port` (`3000`), `protocol` (`http/https`).
- **navigator** : Infos sur le logiciel. Exemples : `language` (`fr-FR`), `cookieEnabled` (`true`), `connection` (`4G`).
- **screen** : Infos sur le matériel. Propriétés : `width`, `height`, `orientation`.

81

DOM

82

- C'est une **API (Application Programming Interface)** regroupant tous les outils nécessaires à la communication entre des langages différents.
 - ▣ HTML et javascript dans notre cas
- Un document HTML est représenté sous forme d'un arbre composé des nœuds (éléments HTML et texte).
- DOM permet de récupérer, éditer, ajouter, cloner, remplacer et supprimer des éléments HTML.

82

Récupérer un élément HTML

83

- Un élément HTML peut être récupéré dans le code javascript avec plusieurs méthodes accessibles à partir de l'objet **document** comme:
 - **getElementById()** prenant comme paramètre l'identifiant de l'élément en HTML
 - **getElementsByName()** prenant comme paramètre la valeur de l'attribut **name** des éléments en HTML
 - **getElementsByClassName()** prenant comme paramètre la valeur de l'attribut **class** des éléments en HTML
 - **getElementsByTagName()** prenant comme paramètre le nom de la balise des éléments en HTML

83

Récupérer un élément HTML

84

Méthode getElementById()

- Elle permet de récupérer l'élément HTML ayant l'identifiant (attribut **id**) passé comme paramètre.

Exemple:

```
<!doctype html>
<html>
  <head>
    <title> getElementById </title>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <a href="#"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      alert(element);
    </script>
  </body>
</html>
```

Cette page indique

[object HTMLDivElement]

OK

84

Récupérer un élément HTML

85

Méthode `getElementById()`

```
<body>

  <div id="groupe">
    <p>Programmation Web</p>
    <a href="#">Lien du cours</a>
  </div>

  <p>Ce paragraphe n'a pas d'ID.</p>

  <script>
    var element = document.getElementById("groupe");
    alert(element);

    // Exemple plus utile : Changer le style ou le contenu de l'élément
    // Changer la couleur de fond du div "groupe" en jaune
    element.style.backgroundColor = "yellow";
  </script>
</body>
```

Programmation Web

[Lien du cours](#)

Ce paragraphe n'a pas d'ID.

85

Récupérer un élément HTML

86

Méthode `getElementsByName()`

- Elle permet de récupérer un **tableau** d'éléments HTML ayant le nom (attribut **name**) passé comme paramètre.
- L'attribut **name** est fréquemment utilisé pour les formulaires (balise **<form>**).
 - ▣ Cases à cocher
 - ▣ Boutons radios

86

Récupérer un élément HTML

87

Méthode `getElementsByName()`

Exemple:

```

<!doctype html>
<html>
  <head>
    <title> getElementByName </title>
  </head>
  <body>
    <form>
      <input type="radio" name="cours" value="pw"> Programmation Web
      <input type="radio" name="cours" value="tdp"> Technique de pointe
    </form>
    <script>
      var elements = document.getElementsByName("cours");
      for(var i=0; i<elements.length; i++)
        alert(elements[i]);
    </script>
  </body>
</html>

```

Cette page indique
[object HTMLInputElement]

OK

87

Récupérer un élément HTML

88

Méthode `getElementsByName()`

```

<body>
  <form>
    <input type="radio" name="cours" value="pw" id="radioPW">
    <label for="radioPW">Programmation Web</label><br>

    <input type="radio" name="cours" value="tdp" id="radioTDP">
    <label for="radioTDP">Technique de pointe</label>
  </form>
  <p id="resultat"></p>
  <script>
    // --- Étape 1 : Récupération ---
    // La méthode retourne une NodeList (une collection) de tous les éléments
    // ayant l'attribut name="cours".
    var elements = document.getElementsByName("cours");

    // --- Étape 2 : Traitement ---
    var resultatMessage = "Éléments trouvés (type: [object HTMLInputElement]) :\n";

    // Parcourir la collection d'éléments avec une boucle for
    for (var i = 0; i < elements.length; i++) {
      // L'alerte originale affiche l'objet DOM, donnant [object HTMLInputElement]
      // alert(elements[i]);

      // Pour un affichage plus informatif (sans bloquer la page avec des alertes) :
      resultatMessage += "Élément " + (i + 1) + " : " + elements[i].value + "\n";
    }

    // Afficher l'information dans un paragraphe de la page au lieu d'une alerte
    document.getElementById("resultat").innerText = resultatMessage;
  </script>
</body>

```

- ☐ Programmation Web
- ☐ Technique de pointe

Éléments trouvés (type: [object HTMLInputElement]) :
Élément 1 : pw
Élément 2 : tdp

88

Récupérer un élément HTML

89

Méthode `getElementsByName()`

- Elle permet de récupérer un **tableau** d'éléments de même type (balise passée comme paramètre).

Exemple:

```
<!doctype html>
<html>
  <head>
    <title> getElementsByName
  </head>
  <body>
    <p> Programmation Web </p>
    <p> Techniques de pointe </p>
    <a href="#"> Lien du cours </a>
    <script>
      var elements = document.getElementsByName('p');
      alert('Le nombre de paragraphes = '+elements.length);
    </script>
  </body>
</html>
```

Cette page indique
Le nombre de paragraphes = 2

OK

89

Récupérer un élément HTML

90

Méthode `getElementsByClassName()`

- Elle permet de récupérer un **tableau** d'éléments appartenant à la même classe (attribut **class**) passée comme paramètre.

Exemple:

Programmation Web
Techniques de pointe
Lien du cours

```
<!doctype html>
<html>
  <head>
    <title> getElementsByClassName
  </head>
  <body>
    <p class="pg"> Programmation Web </p>
    <p> Techniques de pointe </p>
    <a href="#" class="pg"> Lien du cours </a>
    <script>
      var elements = document.getElementsByClassName('pg');
      for(var i=0; i<elements.length; i++){
        elements[i].style.color="red";
      }
    </script>
  </body>
</html>
```

90

Éditer un élément HTML

91

- Une fois un élément HTML récupéré dans le code javascript, on peut l'éditer de deux façons avec :
 - ▣ Les deux méthodes **getAttribute()** et **setAttribute()** appliquées à l'élément récupéré
 - La méthode **getAttribute()** prend comme paramètre l'attribut déclaré en HTML
 - La méthode **setAttribute()** prend comme paramètres l'attribut déclaré en HTML et la nouvelle valeur affectée
 - ▣ Les **propriétés** qui jouent le rôle de getters et setters en même temps (généralement les attributs en HTML)

91

Éditer un élément HTML

92

- Exemple: **getAttribute()** et **setAttribute()**

```
<!DOCTYPE html>
<html>
<head>
  <title>getAttribute/setAttribute</title>
</head>
<body>
  <p class="pg"> Programmation Web </p>
  <p class="pg"> Techniques de pointe </p>
  <a href="#" id="lien"> Lien du cours </a>
  <script>
    // Sélection de l'élément par son ID
    var link = document.getElementById("lien");
    // Récupération de la valeur actuelle de l'attribut "href"
    var ref = link.getAttribute("href");
    alert(ref);
    // Modification de l'attribut "href" vers une nouvelle URL
    link.setAttribute("href", "http://www.isimm.rnu.tn");
    // Affichage de la nouvelle valeur pour l'attribut "href"
    alert(link.getAttribute("href"));
  </script>
</body>
</html>
```

Cette page indique

#

OK

Cette page indique

http://www.isimm.rnu.tn

OK

92

Éditer un élément HTML

93

□ Exemple: Propriétés

```

<!DOCTYPE html>
<html>
<head>
  <title> Propriétés </title>
  <meta charset="utf-8"/>
</head>
<body>
  <p class="pg"> Programmation Web </p>
  <p class="pg"> Techniques de pointe </p>
  <a href="#" id="lien"> Lien du cours </a>
  <script>
    // Sélection de l'élément par son ID
    var link = document.getElementById("lien");
    // Accès direct à la propriété .href pour lecture
    var ref = link.href;
    alert(ref);
    // Modification directe de la propriété .href
    link.href = "http://www.isimm.rnu.tn"
    // Affichage de la nouvelle valeur
    alert(link.href);
  </script>
</body>
</html>

```

Cette page indique

file:///D:/Enseignement%202025-2026/
Technologies%20et%20Programmation%20WEB%20L2%20INFO/
test.html#

OK

Cette page indique

http://www.isimm.rnu.tn/

OK

93

Éditer un élément HTML

94

Méthode / Propriété	Utilisation	Retourne
link.href (Propriété)	Rapide. Lit/écrit la propriété JS de l'URL (souvent complète).	Une chaîne de caractères
getAttribute("href")	Pour lire la valeur exacte de l'attribut HTML tel qu'écrit.	Une chaîne de caractères
setAttribute("href",...)	Pour écrire la valeur exacte de l'attribut HTML.	Rien (undefined)

94

Éditer un élément HTML

95

Propriétés **className** et **htmlFor**

- Les deux propriétés **class** et **for** sont réservées et remplacées par **className** et **htmlFor** dans le code javascript.
 - ▣ L'attribut **class** en HTML permet de définir le style d'un élément
 - ▣ L'attribut **for** en HTML permet de définir l'étiquette (balise **<label>**) associée à un élément du formulaire
- Ou bien, on peut utiliser les deux méthodes **getAttribute()** et **setAttribute()**.

95

Éditer un élément HTML

96

□ Exemple: Propriété **className**

```

<!doctype html>
<html>
  <head>
    <title> Propriété className </title>
    <meta charset="utf-8"/>
    <style>
      .nvpq {
        color: cyan;}
    </style>
  </head>
  <body>
    <p class="pg"> Programmation Web </p>
    <p> Techniques de pointe </p>
    <a href="#"> Lien du cours </a>
    <script>
      var elements = document.getElementsByClassName("pg");
      for(var i=0;i<=elements.length;i++){
        elements[i].className = 'nvpq';
      }
    </script>
  </body>
</html>

```

Programmation Web
Techniques de pointe
[Lien du cours](#)

96

Éditer un élément HTML

97

Exemple: Propriété **htmlFor**

```
<!doctype html>
<html>
  <head>
    <title> Propriété htmlFor </title>
    <meta charset="utf-8"/>
  </head>
  <body>
    <form>
      <label for = "nom" id = "lab"> Candidat: </label>
      <input type = "text" id = "nom" placeholder = "nom"/> <br>
      <input type = "text" id = "prenom" placeholder = "prénom"/>
    </form>
    <script>
      var element = document.getElementById("lab");
      alert(element.htmlFor);
    </script>
  </body>
</html>
```

Cette page indique
nom

OK

97

Éditer un élément HTML

98

Propriété **innerHTML**

- Elle permet de récupérer, éditer et ajouter du code HTML d'un élément.
- Exemple: récupérer le code HTML d'un élément

```
<!doctype html>
<html>
  <head>
    <title> Propriété innerHTML </title>
    <meta charset = "utf-8"/>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <p> Techniques de pointe </p>
      <a href = "#"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      alert(element.innerHTML);
    </script>
  </body>
</html>
```

Cette page indique

<p> Programmation Web </p>
<p> Techniques de pointe </p>
 Lien du cours

OK

98

Éditer un élément HTML

99

Propriété innerHTML

- Exemple: éditer le code HTML d'un élément

```
<!doctype html>
<html>
  <head>
    <title> Propriété innerHTML </title>
    <meta charset = "utf-8"/>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <p> Techniques de pointe </p>
      <a href = "#"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      element.innerHTML = "<h1> DS </h1>" +
        "<h1> Examen </h1>";
    </script>
  </body>
</html>
```

DS
Examen

99

Éditer un élément HTML

100

Propriété innerHTML

- Exemple: ajouter du code HTML à un élément

```
<!doctype html>
<html>
  <head>
    <title> Propriété innerHTML </title>
    <meta charset = "utf-8"/>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <p> Techniques de pointe </p>
      <a href = "#"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      element.innerHTML += "<h1> DS </h1>" +
        "<h1> Examen </h1>";
    </script>
  </body>
</html>
```

Programmation Web
Techniques de pointe
[Lien du cours](#)
DS
Examen

100

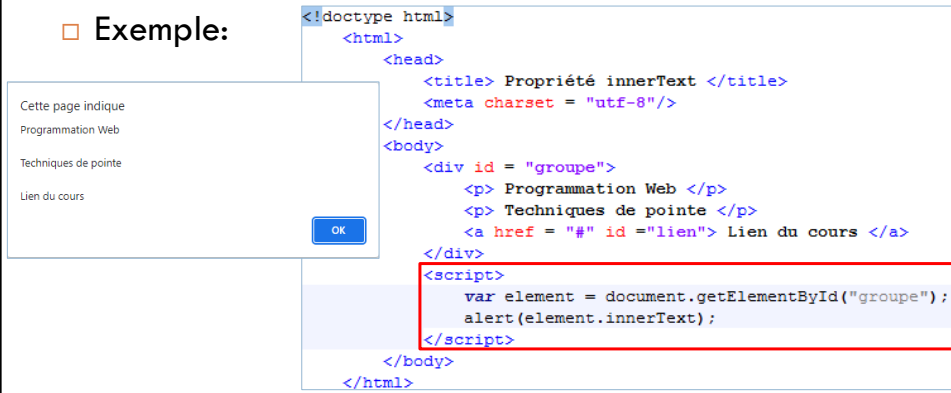
Éditer un élément HTML

101

Propriétés innerText/textContent

- Elles permettent de récupérer et modifier le **texte** contenu dans un élément HTML (sans inclure les balises HTML).

Exemple:



The screenshot shows a web browser window with a page titled "Propriété innerText". The page content includes a list of links: "Cette page indique", "Programmation Web", "Techniques de pointe", and "Lien du cours". A script is shown in the background, which uses `document.getElementById("groupe").innerText` to retrieve the text content of a div with the id "groupe". The script is highlighted with a red box.

```
<!doctype html>
<html>
  <head>
    <title> Propriété innerText </title>
    <meta charset = "utf-8"/>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <p> Techniques de pointe </p>
      <a href = "#" id = "lien"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      alert(element.innerText);
    </script>
  </body>
</html>
```

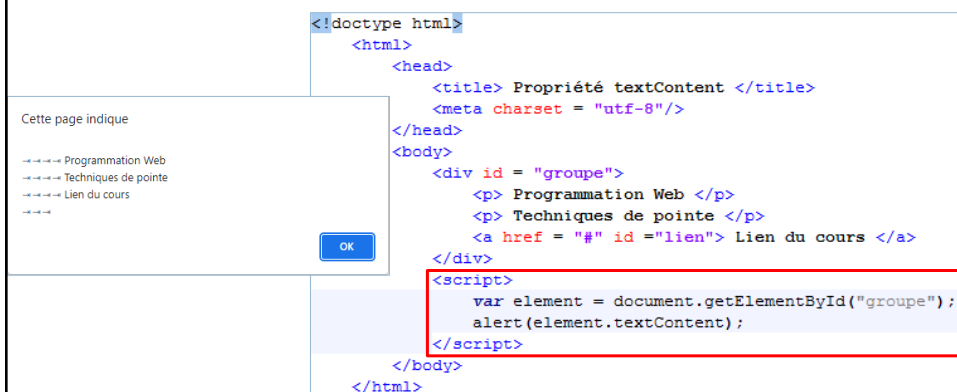
101

Éditer un élément HTML

102

Propriétés innerText/textContent

Exemple: textContent



The screenshot shows a web browser window with a page titled "Propriété textContent". The page content includes a list of links: "Cette page indique", "Programmation Web", "Techniques de pointe", and "Lien du cours". A script is shown in the background, which uses `document.getElementById("groupe").textContent` to retrieve the text content of a div with the id "groupe". The script is highlighted with a red box.

```
<!doctype html>
<html>
  <head>
    <title> Propriété textContent </title>
    <meta charset = "utf-8"/>
  </head>
  <body>
    <div id = "groupe">
      <p> Programmation Web </p>
      <p> Techniques de pointe </p>
      <a href = "#" id = "lien"> Lien du cours </a>
    </div>
    <script>
      var element = document.getElementById("groupe");
      alert(element.textContent);
    </script>
  </body>
</html>
```

102

```
<p id="demo">Bonjour <span style="display:none">caché</span> le monde !
</p>
<script>
  alert(document.getElementById("demo").textContent);
  // Résultat : "Bonjour caché le monde !"
</script>
```

Bonjour caché le monde !

OK

```
<p id="demo">Bonjour <span style="display:none">caché</span> le monde !
</p>
<script>
  alert(document.getElementById("demo").innerText);
  // Résultat : "Bonjour le monde !"
</script>
```

Bonjour le monde !

OK

103

103

Ajouter un élément HTML

104

- L'ajout d'un élément dans un document HTML avec javascript nécessite trois étapes:
 - ▣ Créer l'élément
 - La méthode **createElement()** est utilisée
 - ▣ Configurer les attributs de l'élément
 - Les propriétés sont utilisées (généralement les attributs en HTML)
 - ▣ Insérer l'élément dans le document
 - La méthode **appendChild()** est utilisée

104

Ajouter un élément HTML

105

Étape 1 - Création de l'élément

- La méthode **createElement()** appliquée à l'objet **document** prend comme paramètre la balise associée à l'élément à créer.
- Exemple: création d'un lien hypertexte

```
<!doctype html>
<html>
  <head>
    <title> Ajout élément </title>
    <meta charset = "utf-8"/>
  </head>
  <body id = "b">
    <script>
      var lien = document.createElement("a");
    </script>
  </body>
</html>
```

105

Ajouter un élément HTML

106

Étape 2 - Configuration de l'élément

- Les propriétés permettent de récupérer/modifier les attributs d'un élément HTML.
- Exemple: configuration d'un lien hypertexte

```
<!doctype html>
<html>
  <head>
    <title> Ajout élément </title>
    <meta charset = "utf-8"/>
  </head>
  <body id = "b">
    <script>
      var lien = document.createElement("a");
      lien.id = "hlien";
      lien.href = "http://www.google.tn";
      lien.title = "Accéder à Google";
    </script>
  </body>
</html>
```

106

Ajouter un élément HTML

107

Étape 3 - Insertion de l'élément

- La méthode **appendChild()** appliquée à un **objet parent** prend comme paramètre l'élément à insérer.
- Exemple: insertion d'un lien hypertexte

```

<!doctype html>
<html>
  <head>
    <title> Ajout élément </title>
    <meta charset = "utf-8"/>
  </head>
  <body id = "b">
    <script>
      var lien = document.createElement("a");
      lien.id = "hlien";
      lien.href = "http://www.google.fr";
      lien.title = "Accéder à Google";
      document.getElementById("b").appendChild(lien);
    </script>
  </body>
</html>

```

107

Ajouter un élément HTML

108

Étape 3 - Insertion de l'élément

- C'est insuffisant !!!
- Il faut créer le texte de l'élément avec la méthode **createTextNode()** appliquée à l'objet **document** et prenant comme paramètre le texte à afficher sur la page.
- Il faut appliquer la méthode **appendChild()** à l'élément inséré et prenant comme paramètre l'objet texte à afficher.

108

Ajouter un élément HTML

109

Étape 3 - Insertion de l'élément

- Exemple: insertion d'un lien hypertexte

```
<!doctype html>
<html>
  <head>
    <title> Ajout élément </title>
    <meta charset = "utf-8"/>
  </head>
  <body id = "b">
    <script>
      var lien = document.createElement("a");
      lien.id = "hlien";
      lien.href = "http://www.google.tn";
      lien.title = "Accéder à Google";
      var text = document.createTextNode("Aller à Google");
      lien.appendChild(text);
      document.getElementById("b").appendChild(lien);
    </script>
  </body>
</html>
```

[Aller à Google](http://www.google.tn)

109

Exercice

110

- Écrire le code HTML permettant de créer la page Web suivante:

HTML dynamique avec JS

1. Cliquez pour voir le message
2. IA
3. Cliquez pour changer ma couleur
4. Cliquez pour changer mon texte

110

Exercice

111

- Écrire le code javascript dans un fichier externe permettant de:
 1. Afficher un message d'alerte si on clique sur le premier élément de la liste
 2. Modifier la couleur du texte en rouge du troisième élément de la liste si on clique sur ce dernier
 3. Modifier le texte du quatrième élément de la liste en « Nouveau Texte » si on clique sur ce dernier

111

Exercice

112

- Écrire le code javascript dans un fichier externe permettant de:
 4. Modifier la couleur de tous les éléments de la liste en rouge si on clique sur le premier bouton
 5. Ajouter un élément à la liste ayant le texte « Nouvel élément » si on clique sur le deuxième bouton
 6. Supprimer tous les éléments de la liste si on clique sur le troisième bouton

112